

Mini Paper — 1.4 Data Types, Data Structures & Boolean Algebra

Time: ~35 minutes · Total: 30 marks · Answer all questions.

Mark your work against the mark scheme at the end; aim for ~1 minute per mark. Show all working — method marks are available.

Questions

Q1. Convert the denary number **150** into: (a) 8-bit binary **[1]** (AO2) (b) hexadecimal **[2]** (AO2) Show your working.

Q2. Convert the hexadecimal number **2F** into denary. Show your working. **[2]** (AO2)

Q3. Using **8-bit two's complement**, calculate **45 – 58**. You must: (a) show 45 and 58 in 8-bit binary **[1]** (AO2) (b) form the two's complement of 58 **[1]** (AO2) (c) perform the addition and state the final denary result **[2]** (AO2) **[4 total]**

Q4. A floating point value is shown as **0.00110 × 2³** (mantissa with the point after the sign bit, exponent in denary). (a) Normalise the mantissa and state the new exponent. **[2]** (AO2) (b) Explain *why* numbers are normalised. **[1]** (AO1) **[3 total]**

Q5. The 8-bit signed value **1111 0100** represents **–12** in two's complement. (a) Apply an **arithmetic shift right by 1** and write the resulting bit pattern. **[1]** (AO2) (b) State the denary value of the result and explain why an *arithmetic* (not logical) shift is required here. **[2]** (AO2) **[3 total]**

Q6. Simplify the Boolean expression $\neg(A + \bar{B}) + A \cdot B$ using De Morgan's law and Boolean identities. Show each step. **[4]** (AO2)

Q7. Complete the truth table for the expression $F = A \cdot \bar{B} + \bar{A} \cdot B$ and state which single logic gate it is equivalent to. **[3]** (AO2)

A	B	F
0	0	
0	1	
1	0	
1	1	

Q8. A queue is implemented as a **circular queue** in an array of size 5 (indices 0–4), initially empty with **front = 0** and **rear = 0**. The operations below are applied in order: enqueue P, enqueue Q, enqueue R, dequeue, enqueue S, enqueue T. (a) Show the array contents and the values of **front** and **rear** after all operations. **[3]** (AO3) (b) State **one** advantage of a circular queue over a linear queue. **[1]** (AO1) **[4 total]**

Q9. The values **50, 30, 70, 20, 40, 60** are inserted, in that order, into an initially empty **binary search tree**. (a) Draw or describe the resulting tree. **[2]** (AO3) (b) State the order in which the values are visited by an **in-order** traversal, and what property of the BST this demonstrates. **[2]** (AO3) **[4 total]**

Mark scheme

Q1. [1+2 = 3] (a) $150 = 128 + 16 + 4 + 2 \rightarrow 1001\ 0110$ (1) (b) Split into nibbles: $1001 = 9$, $0110 = 6 \rightarrow 0x96$ (1 for nibble split, 1 for correct answer = 2) Check: $9 \times 16 + 6 = 144 + 6 = 150$ ✓ Examiner tip: always group binary into nibbles from the RIGHT before converting to hex.

Q2. [2] $2F \rightarrow$ first digit $2 = (2 \times 16) = 32$ (1); second digit $F = 15 \rightarrow 32 + 15 = 47$ (1) Examiner tip: multiply the left hex digit by 16, then add the right digit's value ($A-F = 10-15$).

Q3. [4] (a) $45 = 0010\ 1101$; $58 = 0011\ 1010$ (1) (b) -58 : invert $0011\ 1010 \rightarrow 1100\ 0101$, add 1 $\rightarrow 1100\ 0110$ (1) (c) Add $0010\ 1101 + 1100\ 0110$:

```

  0 0 1 0 1 1 0 1   (45)
+ 1 1 0 0 0 1 1 0   (-58)
-----
  1 1 1 1 0 0 1 1

```

No carry out of the MSB. (1) Result $1111\ 0011$ is negative (MSB = 1); magnitude = invert+1 = $0000\ 1101 = 13$, so answer = -13 (1). ($45 - 58 = -13$ ✓) Examiner tip: subtract by adding the two's complement; remember to discard any carry out of the MSB.

Q4. [3] (a) Mantissa must start $0.1...$; currently 0.00110 , so shift the point **right by 2** places $\rightarrow 0.110$ (1). Moving the point right by 2 reduces the exponent by 2: $3 - 2 = 1$, giving 0.110×2^1 (1). (b) Normalising removes wasted leading zeros so the mantissa holds the **maximum significant figures / greatest precision** for the available bits (1). Examiner tip: moving the point right **DECREASES** the exponent; the overall value must stay the same.

Q5. [3] (a) Arithmetic shift right by 1 on $1111\ 0100$: shift right and copy the sign bit (1) in from the left $\rightarrow 1111\ 1010$ (1). (b) $1111\ 1010 = -(\text{invert}+1) = -0000\ 0110 = -6$ (1). An arithmetic shift **preserves the sign bit**, so a negative two's complement number stays negative and is correctly divided by 2; a logical shift would insert a 0, corrupting the sign and giving a wrong (positive) result (1). Examiner tip: $-12 \div 2 = -6$ — state both the bit pattern and the denary check.

Q6. [4] - De Morgan's on the first term: $\neg(A + \bar{B}) = \neg A \cdot \neg(\bar{B}) = \bar{A} \cdot B$ (double negation of $\bar{B} \rightarrow B$) (1 for De Morgan's, 1 for double negation) - Expression becomes $(\bar{A} \cdot B) + (A \cdot B)$ (1) - Factor out B (distributive): $B \cdot (\bar{A} + A) = B \cdot 1 = B$ (1) Final answer: **B** Examiner tip: with De Morgan's, change the operator AND negate each term, then look for a common factor to absorb.

Q7. [3]

A	B	F
0	0	0
0	1	1
1	0	1
1	1	0

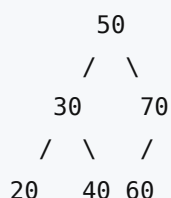
(1 mark per correct pair of rows = up to 2 for the column.) F is 1 only when the inputs **differ**, so it is equivalent to an **XOR** gate (1). Examiner tip: $A \cdot \bar{B} + \bar{A} \cdot B$ is the standard sum-of-products definition of XOR.

Q8. [4] (a) Tracing (size 5, MOD 5; rear points to next free slot):

Operation	Array (idx 0–4)	front	rear
enqueue P	[P, , , ,]	0	1
enqueue Q	[P, Q, , ,]	0	2
enqueue R	[P, Q, R, ,]	0	3
dequeue	[, Q, R, ,]	1	3
enqueue S	[, Q, R, S,]	1	4
enqueue T	[, Q, R, S, T]	1	0

Final array contains **Q, R, S, T** (index 0 free), **front = 1, rear = 0** (1 for correct contents, 1 for front, 1 for rear = 3). (b) A circular queue **reuses the space freed by dequeuing** (rear wraps round with MOD), so it does not waste slots at the front like a linear queue (1). *Examiner tip: show the MOD wraparound — rear goes 4 → 0 once it passes the end of the array.*

Q9. [4] (a) BST built by inserting 50, 30, 70, 20, 40, 60:



(1 for 50 as root with 30 left / 70 right; 1 for 20,40 under 30 and 60 as left child of 70) (b) In-order (left, root, right) visits: **20, 30, 40, 50, 60, 70** (1). This is **ascending sorted order**, demonstrating the BST ordering property (left subtree < node < right subtree) (1). *Examiner tip: an in-order traversal of any BST always outputs the values in ascending order — a quick way to check your tree.*